

Royaume du Maroc - Ministère de l'Éducation Nationale, du Prescolaire et des Sports

Académie Régionale de l'Éducation et de la Formation Draa-Tafilalet

Direction Provinciale de Ouarzazate

Rapport de Projet de Fin d'Études

Deuxième année BTS - Développement des Applications Informatiques

Conception et réalisation d'une application mobile de gestion des commandes d'un restaurant

Champ	Valeur
Projet	Mon PF App
Réalisé par	HAJJI Yasmine
Établissement	Lycee Qualifiant Technique Ibn Al-Haitam
Encadrement	Encadrant pédagogique : à compléter
Année de formation	2024 - 2026

Remerciements

Je remercie l'equipe pedagogique du BTS Developpement des Applications Informatiques du Lycee Qualifiant Technique Ibn Al-Haitam pour l'encadrement, les connaissances techniques et la methodologie acquises durant la formation.

Je remercie egalement toutes les personnes qui ont participe a la definition du besoin, aux tests, aux retours d'utilisation et a l'amelioration progressive de l'application Mon PF App.

Ce projet a permis de rapprocher les acquis de programmation, base de donnees, analyse UML, securite et documentation d'un besoin applicatif concret: la gestion des commandes d'un restaurant.

Resume

Ce rapport presente Mon PF App, une application mobile de gestion des commandes et des livraisons d'un restaurant. Le projet repond a la problematique de la gestion manuelle: erreurs de commande, retards de preparation, manque de suivi et communication insuffisante entre client, restaurant et livreur.

La solution s'appuie sur Flutter pour l'application mobile et Laravel pour l'API backend. Le systeme permet au client de consulter le menu, ajouter des plats au panier, passer une commande et suivre son statut. Il propose aussi des espaces dedies a l'administrateur, au serveur et au livreur.

La version livree est une version demo-prod: elle dispose d'une interface aboutie, d'un backend local fonctionnel, d'une base SQLite seedee, de tests automatiques et d'une documentation permettant d'installer, tester, debugger et presenter le projet.

Table des matieres

- 1 Introduction generale
- 2 Contexte et problematique
- 3 Objectifs du projet
- 4 Cahier des charges
- 5 Acteurs du systeme
- 6 Analyse fonctionnelle
- 7 Conception des donnees
- 8 Architecture technique
- 9 Backend Laravel
- 10 Application Flutter
- 11 Securite
- 12 Algorithmes et regles metier
- 13 Interface et experience utilisateur
- 14 Tests et validation
- 15 Mise en service locale
- 16 Android et Windows
- 17 Difficultes et solutions
- 18 Validation du cahier de charge
- 19 Perspectives
- 20 Conclusion
- 21 Annexes

1. Introduction generale

Dans le cadre de la formation BTS Developpement des Applications Informatiques, le projet de fin d'etudes doit demontrer la capacite a analyser un besoin, concevoir une solution, realiser une application, tester son fonctionnement et documenter le travail effectue.

Mon PF App est une application de gestion des commandes d'un restaurant. Elle montre un parcours client complet et des espaces operationnels pour l'administration et la livraison.

Le choix du sujet est pertinent pour un PFE car il mobilise plusieurs competences: analyse des acteurs, modelisation des donnees, interface mobile, API REST, securite, tests, build Android et documentation professionnelle.

Element	Valeur
Formation	BTS Developpement des Applications Informatiques
Projet	Mon PF App
Cible principale	Android et Windows
Type	Application mobile + backend API

2. Contexte et problematique

Les restaurants gerent de plus en plus de commandes a emporter et en livraison. Lorsque les commandes sont prises manuellement, les erreurs de saisie et les oublis deviennent frequents.

Le client souhaite connaitre l'etat de sa commande. Le restaurant doit organiser les preparations. Le livreur doit savoir quelles commandes sont disponibles et lesquelles lui sont affectees.

La problematique du projet est donc: comment digitaliser le processus de commande afin de reduire les erreurs humaines, acclerer le traitement et ameliorer la communication entre les acteurs ?

Probleme	Consequence	Reponse Mon PF App
Erreur de commande	Plat incorrect ou quantite fausse	Panier numerique et validation
Retard	Client insatisfait	Suivi par statut
Mauvaise communication	Livreur ou restaurant mal informe	Espaces dedies
Menu difficile a maintenir	Prix/disponibilite non a jour	Catalogue API et CRUD admin

3. Objectifs du projet

L'objectif general est de realiser une application mobile de gestion des commandes d'un restaurant. Elle doit organiser le processus depuis le choix des plats jusqu'a la livraison.

Les objectifs specifiques consistent a permettre au client de consulter le menu, de passer une commande et de suivre son etat; a permettre au restaurant de gerer les commandes; et a permettre au livreur de consulter et accepter les livraisons.

Sur le plan technique, le projet doit etre structure, maintenable et testable. Le repository a donc ete organise avec un frontend Flutter, un backend Laravel, des documents de specification, des guides et des scripts.

- Digitaliser la commande client.
- Centraliser le menu et les commandes.
- Gerer les roles client, admin, serveur et livreur.
- Construire un APK Android testable.
- Documenter installation, debug et soutenance.

4. Cahier des charges

Le cahier de charge fourni indique les besoins fonctionnels cote client, restaurant et livreur. Il indique aussi les besoins non fonctionnels: securite, simplicite, rapidite, fiabilite et maintenance.

La version actuelle couvre le coeur du cahier de charge. Le client peut se connecter, consulter le menu, ajouter au panier, passer une commande et suivre son statut. Le livreur peut consulter la file et accepter une livraison. L'administrateur dispose de statistiques et d'une vision commandes.

Certaines evolutions restent necessaires pour une vraie production commerciale: hebergement HTTPS, base hebergee, notifications, paiement et interface CRUD admin complete.

Besoin	Etat
Connexion/inscription	Couvert demo + API
Menu	Couvert demo + API
Panier/commande	Couvert avec POST /api/orders
Suivi	Couvert visuellement; temps reel a ameliorer
Livreur	File et acceptation disponibles
Admin	Stats et endpoints disponibles

5. Acteurs du systeme

Le systeme est concu autour de quatre acteurs. Chaque acteur a un role precis afin de respecter les responsabilites metier.

Le client utilise l'application pour commander. L'administrateur gere le systeme global. Le serveur intervient dans le traitement des commandes cote restaurant. Le livreur prend en charge la livraison.

Acteur	Responsabilites
Client	Consulter menu, panier, commander, suivre
Administrateur	Statistiques, commandes, menu, affectation
Serveur	Traitement operationnel des commandes
Livreur	Consulter, accepter, livrer

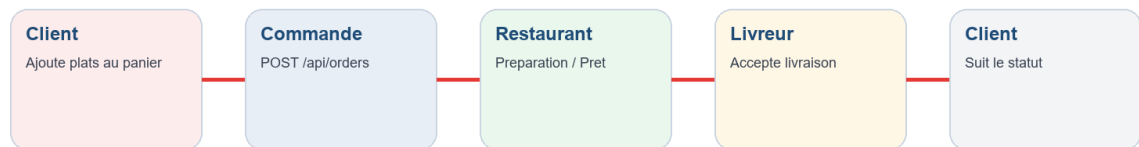
6. Analyse fonctionnelle

L'analyse fonctionnelle décompose le système en cas d'utilisation. Chaque cas d'utilisation correspond à une action visible dans l'application ou dans l'API.

Le parcours principal est le passage de commande. Le client choisit des plats, valide le panier, l'API crée une commande et les espaces opérationnels peuvent ensuite suivre ou traiter cette commande.

Cette analyse a guidé la structure des fichiers Flutter et Laravel: les fonctionnalités sont regroupées par domaine, ce qui facilite la compréhension par Yasmine et par le jury.

Cycle de commande



Règle: une commande ne peut pas être créée sans plats. Les statuts sont contrôlés par l'API.

Illustration - 6. Analyse fonctionnelle

7. Conception des donnees

Le modele de donnees s'organise autour des utilisateurs, categories, plats, commandes et lignes de commande. Cette structure evite de stocker une commande comme simple texte et permet de garder l'historique des prix et quantites.

Le champ role de l'utilisateur permet de distinguer client, admin, serveur et livreur. La commande contient son statut, son adresse, son total et eventuellement le livreur affecte.

SQLite est utilisee pour la demo locale. En production, le meme schema peut etre porte vers MySQL ou PostgreSQL.

Modele logique simplifie

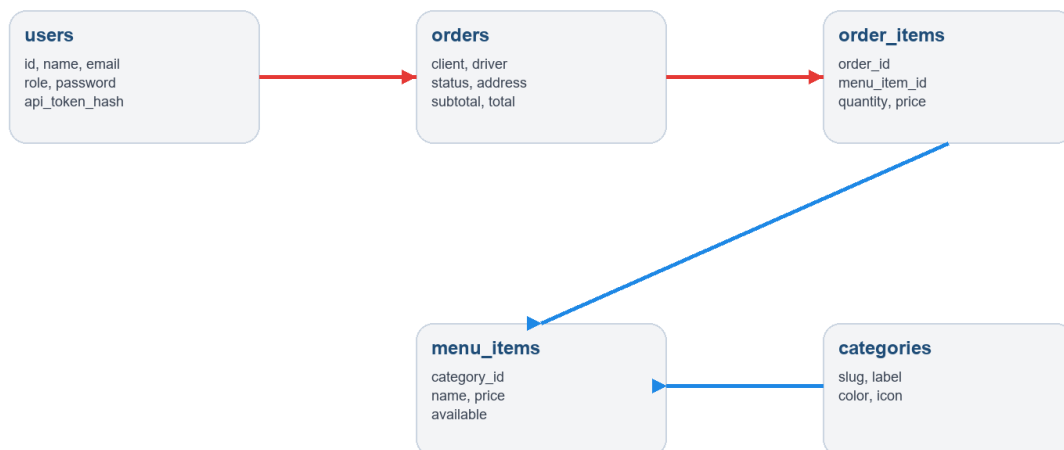


Illustration - 7. Conception des donnees

8. Architecture technique

L'architecture separe le frontend du backend. Flutter gere l'interface, l'etat utilisateur et le panier. Laravel gere les validations, les routes REST et la persistance.

Cette separation est professionnelle: elle permet de faire evoluer l'interface sans changer la base, et de remplacer SQLite par une base hebergee sans modifier toute l'application mobile.

Le mode demo reste disponible pour garantir une presentation meme si le serveur n'est pas accessible. Le mode backend reel s'active avec les variables DEMO_MODE=false et API_BASE_URL.

Architecture demo-prod

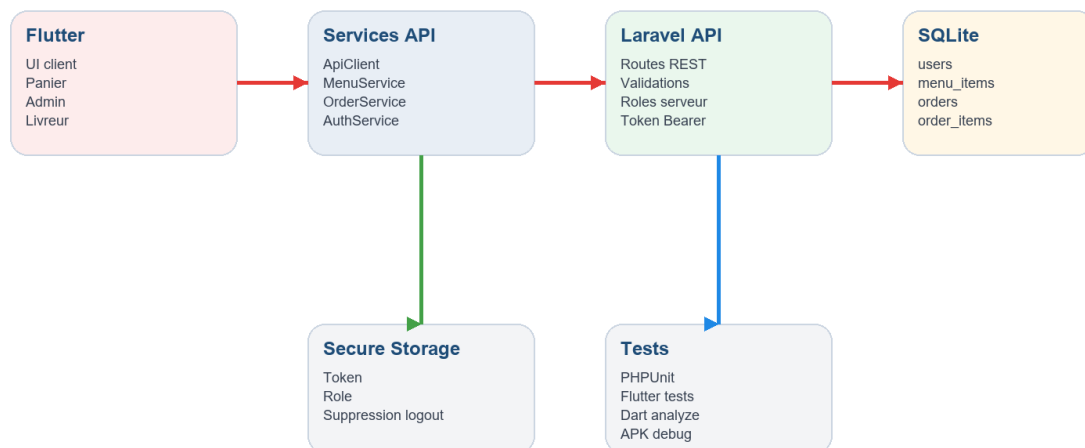


Illustration - 8. Architecture technique

9. Backend Laravel

Le backend se trouve dans mon_pfapi. Il expose des endpoints REST publics et proteges. Les endpoints publics sont /api/health, /api/menu, /api/register et /api/login.

Les routes proteges utilisent un middleware TokenAuth. Le token est envoye dans le header Authorization: Bearer. Le backend stocke seulement un hash du token, ce qui reduit le risque en cas de fuite de base.

Les tests PHPUnit valident le menu seed, la connexion, l'inscription forcee en client et la creation de commande.

Composant	Role
AuthController	Register, login, logout
MenuController	Catalogue et CRUD menu
OrderController	Commandes, statuts, livreur, stats
TokenAuth	Controle token et role
DatabaseSeeder	Donnees demo

10. Application Flutter

L'application Flutter se trouve dans mon_pfapp. Elle contient des écrans pour l'authentification, l'accueil client, le menu, le panier, le suivi, le profil, l'administration et la livraison.

La classe MonPfApp centralise l'état principal: utilisateur connecté, écran courant, menu chargé, commandes et panier. Les services MenuService et OrderService récupèrent les données depuis l'API ou depuis DemoData.

L'interface utilise des images réelles de plats, un logo personnalisé et des transitions animées afin d'obtenir un rendu plus proche d'un produit final.

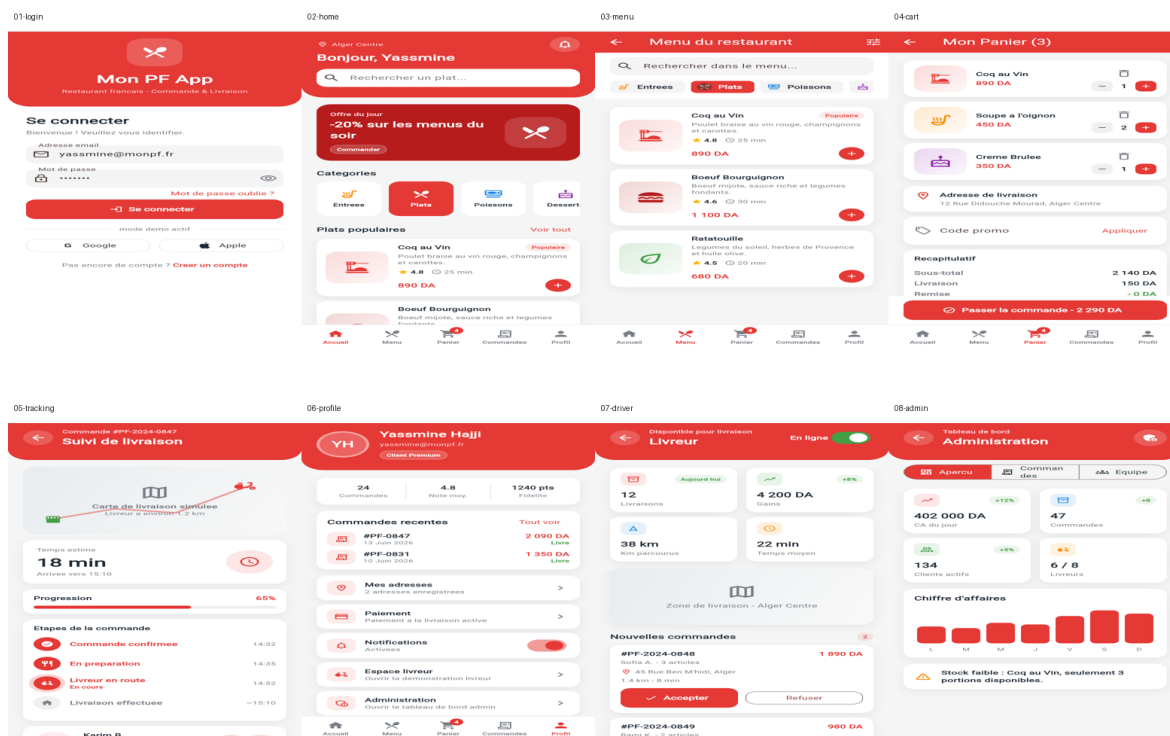


Illustration - 10. Application Flutter

11. Securite

La securite a ete renforcee a plusieurs niveaux. La selection de role a ete retiree de l'inscription publique. Le role final est impose par le backend.

Le token est stocke dans flutter_secure_storage cote application. Cote API, le token brut n'est pas conserve: seul son hash SHA-256 est stocke.

L'APK release garde une politique reseau stricte. HTTP local est autorise seulement pour debug/profile afin de faciliter les tests sur Wi-Fi pendant la soutenance.

Risque	Traitement
Role public admin	Role client force cote serveur
Token local	Secure storage
Token base	Hash du token
HTTP release	Refuse sauf debug/profile
Serveur lent	Timeout et erreurs

12. Algorithmes et regles metier

Le projet repose sur des algorithmes applicatifs simples et defendables. Le panier verifie si un plat existe deja; si oui, il augmente la quantite, sinon il ajoute une nouvelle ligne.

Le total est calcule par addition des lignes: prix unitaire multiplie par quantite, puis ajout des frais de livraison. La creation de commande refuse un panier vide.

Le cycle de statut permet de représenter l'avancement: preparation, pret, en route, livre ou annule. Les roles controlent les statuts qu'ils peuvent modifier.

Regle	Explication
Panier non vide	Une commande exige au moins une ligne
Quantite	Borne entre 1 et 99 cote app
Total	Somme lignes + livraison
Role	Autorisation cote middleware
Statut	Normalisation cote backend

13. Interface et experience utilisateur

L'interface a ete inspiree par la maquette restaurant fournie. Elle privilegie une presentation mobile claire: header rouge, cartes blanches, navigation basse, visuels de plats et boutons d'action visibles.

L'objectif UI/UX est de permettre au client de comprendre rapidement le parcours. Le menu est filtre par categorie, le panier resume le total et le suivi affiche les etapes de livraison.

Les espaces admin et livreur utilisent des cartes de statistiques et des listes de commandes afin de rester lisibles sur telephone.

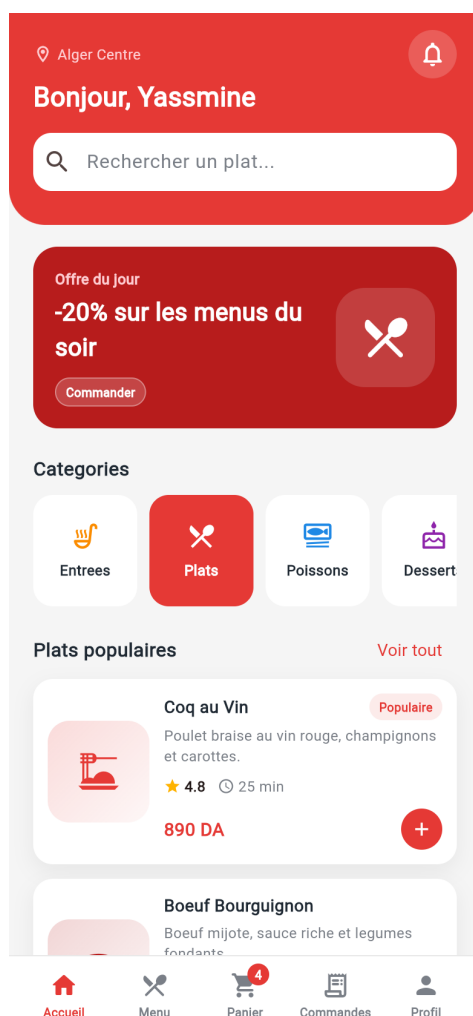


Illustration - 13. Interface et experience utilisateur

14. Tests et validation

Les tests automatiques prouvent que l'application n'est pas seulement une maquette. Flutter teste les écrans critiques et Laravel teste les flux API.

Les commandes validées sont dart analyze, flutter test et php artisan test. Un test HTTP live a confirmé qu'un client peut se connecter et créer une commande via l'API.

Le build Android debug a été généré et pointé vers l'API locale du PC. Le build Windows reste bloqué par ATL, un composant externe Visual Studio.

Contrôle	Résultat
Dart analyze	No issues found
Flutter tests	5 tests passed
Laravel tests	5 tests passed
API live	Commande créée
APK debug	Construit
Windows	ATL manquant

15. Mise en service locale

La mise en service locale se fait en deux étapes: demarrer Laravel puis lancer Flutter avec l'URL API. Le backend doit rester ouvert pendant le test.

Pour Android physique, le telephone doit etre sur le meme Wi-Fi que le PC, et l'URL doit utiliser l'adresse IPv4 Wi-Fi du PC au lieu de 127.0.0.1.

Les commandes exactes sont documentees dans le tutoriel technique et le runbook demo-prod.

```
cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapi"
..\tools\runtime\php-8.4.22\php.exe artisan serve --host=0.0.0.0 --port=8000

cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp"
C:\Users\LOQ\dev\flutter\bin\flutter.bat run --dart-define=DEMO_MODE=false
--dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

16. Android et Windows

Android est la cible la plus avancée. L'APK debug existe et peut être installée avec ADB lorsque le téléphone est connecté et autorisé.

Windows est une cible prioritaire mais le build nécessite l'installation du composant Visual Studio ATL. L'erreur exacte est `atlstr.h` introuvable.

Cette distinction doit être expliquée clairement au jury: le code Windows n'est pas bloqué par Flutter, mais par une dépendance système manquante.

Cible	Etat	Action
Android	APK debug prêt	Installer via ADB
Windows	ATL manquant	Installer composant Visual Studio
Release Android	HTTPS requis	Déployer API HTTPS

17. Difficultes et solutions

Plusieurs difficultes techniques ont ete rencontrees pendant la transformation demo en demo-prod. Chaque difficulte a ete diagnostiquee puis contournee proprement.

PHP et Composer n'etaient pas installes globalement. Chocolatey etait bloque par des permissions Windows. La solution a ete une installation portable locale dans tools/runtime.

Le wrapper dart.bat etait lent dans ce contexte. L'analyse est donc lancee avec dart.exe direct. Le build Android a aussi necessite une correction du manifest debug pour autoriser HTTP local.

Difficulte	Solution
Chocolatey bloque	PHP/Composer portable
dart.bat lent	dart.exe direct
Cleartext Android	tools:replace debug/profile
ADB non trouve	Chemin complet platform-tools
Windows ATL	Installer composant Visual Studio

18. Validation du cahier de charge

La version actuelle valide les besoins essentiels du cahier de charge pour une demonstration PFE. Les parcours principaux sont presents et connectes a une API locale.

La validation complete production demanderait toutefois un serveur HTTPS heberge, une base MySQL/PostgreSQL et une synchronisation temps reel. Ces limites sont normales pour une version demo-prod.

Exigence	Validation
Client menu/panier/commande	Oui
Suivi commande	Oui, visuel + statut
Admin statistiques	Oui
Livreur file/acceptation	Oui
CRUD menu	API oui, UI a completer
Temps reel	Evolution
HTTPS deploye	Evolution

19. Perspectives

Les perspectives principales concernent la production réelle. L'API devrait être déployée sur un domaine HTTPS et connectée à une base hébergée.

Laravel Sanctum peut remplacer le token simple pour une gestion d'authentification plus standard. Des notifications push peuvent informer le client quand le statut change.

Une interface admin complète pour le CRUD menu permettrait au gérant de modifier plats et disponibilités sans intervention technique.

- Déploiement HTTPS
- Base MySQL/PostgreSQL
- Laravel Sanctum
- Notifications push
- Paiement en ligne
- CI/CD GitHub Actions

20. Conclusion

Mon PF App constitue une base solide de projet de fin d'etudes BTS. Le projet montre la capacite a comprendre un besoin, concevoir une architecture, developper une application, ajouter un backend, securiser les roles et tester les flux.

La version demo-prod est suffisante pour une presentation claire: elle possede un parcours client, un espace admin, un espace livreur, une API Laravel, une base SQLite, un APK Android et une documentation detaillee.

Le travail restant est bien identifie et peut etre presente comme perspectives de production. Cette transparence renforce la credibilite du projet.

Annexe A - Comptes de demonstration

Email	Role	Mot de passe
yasmine@monpf.fr	client	password
admin@monpf.fr	admin	password
serveur@monpf.fr	serveur	password
livreur@monpf.fr	livreur	password

Annexe B - Endpoints API

Methode	Endpoint	Role
GET	/api/health	public
GET	/api/menu	public
POST	/api/register	public
POST	/api/login	public
GET	/api/orders	connecte
POST	/api/orders	client
GET	/api/admin/stats	admin/serveur
GET	/api/driver/orders	livreur/admin

Annexe C - Fichiers importants

Fichier	Role
mon_pfapp/lib/app/mon_pf_app.dart	Etat global et navigation
mon_pfapp/lib/data/api_client.dart	Client HTTP
mon_pfapi/routes/api.php	Routes REST
mon_pfapi/database/seeders/DatabaseSeeder.php	Donnees demo
docs/demo-prod-runbook.md	Guide lancement
docs/backend-api.md	Documentation API